

AWS Deployment Runbook

Complete step-by-step record of how the AEGIS AWS production environment was provisioned. Use this as a reference for re-provisioning, disaster recovery, or adding a second AWS region.

Prerequisites

- AWS account `<AWS_ACCOUNT_ID>` (`aegis-imaging`)
- AWS CLI configured: `aws configure` with `<deploy-iam-user>` IAM user credentials
- Terraform `>= 1.5`: `brew install terraform`
- Docker with `--platform linux/amd64` build support (Rosetta on Apple Silicon)
- GoDaddy DNS access for `aegisimaging.ai`
- ACM (AWS Certificate Manager) access in `us-east-1`

Region & Account

Detail	Value
Region	<code>us-east-1</code>
Account ID	<code><AWS_ACCOUNT_ID></code>
IAM deploy user	<code><deploy-iam-user></code> (access key lives only in your local AWS profile — never commit it)
Terraform state bucket	<code>aegis-prod-terraform-state</code> (S3, versioned, KMS-encrypted)
Terraform lock table	<code>aegis-terraform-locks</code> (DynamoDB)

Phase 1 — Bootstrap Terraform State (one-time, already done)

These commands were run once before `terraform init` :

```
aws s3 mb s3://aegis-prod-terraform-state --region us-east-1
aws s3api put-bucket-versioning \
  --bucket aegis-prod-terraform-state \
  --versioning-configuration Status=Enabled
aws dynamodb create-table \
  --table-name aegis-terraform-locks \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

Terraform import (after bootstrap, to bring existing resources under TF state):

```
terraform import aws_s3_bucket.tf_state aegis-prod-terraform-state
terraform import aws_dynamodb_table.tf_locks aegis-terraform-locks
```

Note: DynamoDB's default SSE (AWS-owned keys) conflicts with the `server_side_encryption` block in Terraform if present. Remove that block — DynamoDB encrypts at rest by default.

Phase 2 — ACM TLS Certificate

`*.aegisimaging.ai` only covers one level of subdomain. Two-level subdomains like `aws.api.aegisimaging.ai` require explicit SANs.

Cert ARN (current): `arn:aws:acm:us-east-1:<AWS_ACCOUNT_ID>:certificate/<ACM_CERT_ID>`

Covered SANs:

- `aegisimaging.ai`
- `*.aegisimaging.ai`
- `aws.api.aegisimaging.ai`
- `aws.admin.aegisimaging.ai`

To add a new subdomain (e.g. `aws.dvw.aegisimaging.ai`):

1. ACM Console → Request certificate → DNS validation
 2. Add all existing SANs **plus** the new one
 3. Add the new DNS validation CNAME to GoDaddy (existing SANs are already validated)
 4. Wait for cert status = **Issued** (~2 min after CNAME propagates)
 5. Update `acm_certificate_arn` in `terraform/aws/terraform.tfvars`
 6. Run `terraform apply` — ALB listener updates to the new cert in ~1 second
-

Phase 3 — Core Infrastructure via Terraform

```
cd terraform/aws
terraform init      # initialises S3 backend + downloads providers
terraform plan      # review what will be created
terraform apply -auto-approve
```

What this provisions:

- VPC `10.0.0.0/16` with public (`10.0.1/24`) + private (`10.0.10/24`) subnets
- Internet Gateway, NAT Gateway, route tables
- KMS key (all encryption at rest)
- S3 bucket for DICOM file storage (private, KMS-encrypted, versioned)
- RDS PostgreSQL 15 in private subnets (SSL required, password auto-rotated in Secrets Manager)
- ECS Fargate cluster `aegis-cluster`
- ECR repositories for all service images
- ALB (internet-facing) with HTTPS listener + ACM cert
- Cognito user pool + app client + hosted domain
- ALB listener rules (host-based + path-based)
- ECS task definitions + services: API, admin-dashboard, sidecars, DWV

- CloudWatch log group
- Service discovery namespace `aegis.local` (for sidecar-to-sidecar DNS)
- SNS + SQS for event notifications
- IAM roles (ECS task execution + task role)

Phase 4 — ALB DNS Architecture

Single ALB `<ALB_DNS_NAME>` serves all subdomains via host-based listener rules:

Priority	Host	Auth	Target
1	<code>aws.api.aegisimaging.ai</code>	None (Go API validates Cognito JWT)	API target group
2	<code>aws.admin.aegisimaging.ai</code>	Cognito authenticate-cognito	Admin target group
3	<code>aws.dvw.aegisimaging.ai</code>	None (public viewer)	DWV target group
10	<code>any, /healthz</code>	None	API target group
20	<code>any, /api/upload/*</code>	None	API target group
30	<code>any, /api/export/*</code>	None	API target group
40	<code>any, /api/contact</code>	None	API target group
50	<code>any, /api/storage/*</code>	None	API target group
60	<code>any, /api/projects</code>	None	API target group
70	<code>any, /api/projects/*/active-anon-profile</code>	None	API target group
80	<code>any, /dicomweb*</code>	None	API target group
90	<code>any, /api/*</code>	Cognito authenticate-cognito	API target group
default	<code>any</code>	Cognito authenticate-cognito	Admin target group

GoDaddy CNAMEs (all → same ALB):

```
aws.api.aegisimaging.ai    → <ALB_DNS_NAME>
aws.admin.aegisimaging.ai → <ALB_DNS_NAME>
```

```
aws.dvw.aegisimaging.ai → <ALB_DNS_NAME>
```

Phase 5 — Auth Architecture (Cognito JWT Flow)

Problem solved: The admin dashboard React app makes relative `/api/*` calls. nginx in the admin container proxies them. The full auth chain:

```
browser → aws.admin.aegisimaging.ai/api/studies
  → ALB (priority 2): Cognito session checked → X-Amzn-0idc-Data JWT injected
  → nginx: proxies /api/ → https://aws.api.aegisimaging.ai/api/
    forwards X-Amzn-0idc-Data header explicitly
  → ALB (priority 1): plain forward (no Cognito)
  → Go API: AUTH_ENABLED=true, AUTH_PROVIDER=aws
    extracts email from JWT (no signature verification – ALB guarantees
    integrity)
    looks up email in admin_users table → 200 OK
```

nginx.conf uses `${API_URL}` template variable (set to `https://aws.api.aegisimaging.ai` via ECS task definition env var). Default value = `https://api.aegisimaging.ai` for GCP.

API env vars:

- `AUTH_ENABLED=true`
 - `AUTH_PROVIDER=aws`
 - `DB_SSLMODE=require` (RDS enforces SSL; GCP Cloud SQL does not)
-

Phase 6 — Build & Push Container Images to ECR

ECR login (required before each push session)

```
aws ecr get-login-password --region us-east-1 \
| docker login --username AWS \
--password-stdin <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com
```

Build & push API

```
docker build --platform linux/amd64 \
-t <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/api:latest \
-f api/Dockerfile api/
docker push <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/api:latest
```

Build & push admin dashboard (AWS-specific: DWV URL baked in)

```
docker build --platform linux/amd64 \
--build-arg VITE_DWV_BASE_URL=https://aws.dvw.aegisimaging.ai \
-t <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/admin-dashboard:latest \
-f frontend/admin-dashboard/Dockerfile .
```

```
docker push <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/admin-  
dashboard:latest
```

Note: the build context is the repo root (.), not frontend/admin-dashboard/ — the Dockerfile copies the root package-lock.json and the client/ workspace package. API_URL is NOT a build arg — it is set at runtime via ECS task definition env var.

Build & push DWV

```
docker build --platform linux/amd64 \  
-t <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/dwv:latest \  
dwv/  
docker push <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/dwv:latest
```

Build & push sidecars (defacing, phi-detection, qc-service, bids-service, classification-service, protocol-service, synth-service)

```
for svc in defacing phi-detection qc-service bids-service classification-service  
protocol-service synth-service; do  
  docker build --platform linux/amd64 \  
  -t <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/${svc}:latest \  
  ${svc}/  
  docker push <AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/${svc}:latest  
done
```

Phase 7 — ECS Service Deployment

After terraform apply creates/updates task definitions, force-deploy each service to pull the latest image:

```
aws ecs update-service \  
--cluster aegis-cluster \  
--service aegis-api \  
--force-new-deployment \  
--region us-east-1  
  
aws ecs update-service \  
--cluster aegis-cluster \  
--service aegis-admin-dashboard \  
--force-new-deployment \  
--region us-east-1  
  
aws ecs update-service \  
--cluster aegis-cluster \  
--service aegis-dwv \  
--force-new-deployment \  
--region us-east-1
```

Monitor rollout:

```
aws ecs describe-services \
  --cluster aegis-cluster \
  --services aegis-api aegis-admin-dashboard aegis-dwv \
  --region us-east-1 \
  --query 'services[*].
{name:serviceName,running:runningCount,desired:desiredCount,status:status}'
```

Phase 8 — Bootstrap Admin User (one-time)

After infrastructure is up, create a Cognito user and register them in the `admin_users` table.

1. Create Cognito user

```
aws cognito-idp admin-create-user \
  --user-pool-id <COGNITO_USER_POOL_ID> \
  --username ops@aegisimaging.ai \
  --user-attributes \
    Name=email,Value=ops@aegisimaging.ai \
    Name=email_verified,Value=true \
  --temporary-password "TempPass123!@#" \
  --region us-east-1
```

2. Log in to admin dashboard

Navigate to `https://aws.admin.aegisimaging.ai`. Enter the temporary password when prompted and set a permanent password.

3. Register in `admin_users` table

With `AUTH_ENABLED=false` (bootstrap only) or via the GCP admin dashboard:

```
curl -s -X POST https://aws.api.aegisimaging.ai/api/admin-users \
  -H "Content-Type: application/json" \
  -d '{"email":"ops@aegisimaging.ai","name":"Matt","role":"admin","enabled":true}'
```

After bootstrapping, set `AUTH_ENABLED=true` in the API task definition.

To create additional Cognito users

```
aws cognito-idp admin-create-user \
  --user-pool-id <COGNITO_USER_POOL_ID> \
  --username NEW_EMAIL \
  --user-attributes Name=email,Value=NEW_EMAIL Name=email_verified,Value=true \
  --temporary-password "TempPass123!@#" \
  --region us-east-1
```

Then register in `admin_users` via the Users tab in the admin dashboard.

Key Resource ARNs & IDs

Resource	Value
VPC	(see terraform output)
ALB DNS	<ALB_DNS_NAME>
ACM cert	arn:aws:acm:us-east-1:<AWS_ACCOUNT_ID>:certificate/<ACM_CERT_ID>
Cognito user pool	<COGNITO_USER_POOL_ID>
Cognito domain	aegis-prod-auth
ECS cluster	aegis-cluster
RDS identifier	(see terraform output)
ECR prefix	<AWS_ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/aegis/

Common Operations

View live API health

```
curl https://aws.api.aegisimaging.ai/healthz | python3 -m json.tool
```

Check ECS service logs

```
aws logs tail /ecs/aegis --follow --region us-east-1
```

Force-redeploy a single service (e.g. after pushing new image)

```
aws ecs update-service --cluster aegis-cluster --service aegis-api \
  --force-new-deployment --region us-east-1
```

List Cognito users

```
aws cognito-idp list-users --user-pool-id <COGNITO_USER_POOL_ID> --region us-east-1
```

Get all ECR repository URLs

```
cd terraform/aws && terraform output ecr_repositories
```

Differences from GCP Deployment

Concern	GCP	AWS
Auth provider	AUTH_PROVIDER=iap	AUTH_PROVIDER=aws
DB SSL	DB_SSLMODE=disable	DB_SSLMODE=require
File storage	STORAGE_MODE=gcs	STORAGE_MODE=s3
Containers	Cloud Run	ECS Fargate
Admin dashboard API URL	https://api.aegisimaging.ai (default)	https://aws.api.aegisimaging.ai (via API_URL env)
DWV URL	https://dwv-<CLOUD_RUN_HASH>-uc.a.run.app (Cloud Run, baked as build arg)	https://aws.dwv.aegisimaging.ai (baked as build arg)
CI/CD	Cloud Build auto-deploys on push to develop	Manual ECR push + ECS force-deploy (no CI yet)
Terraform state	GCS bucket <GCP_PROJECT_ID>-tfstate	S3 bucket aegis-prod-terraform-state

Known Gotchas

ALB Target Group Port vs Container Port

Docker Compose `ports: "HOST:CONTAINER"` shows host-side mapping. ECS `containerPort` must be the **container** port (`8080`), not the host port. The DWV docker-compose maps `3005:8080` — the container listens on `8080` .

`envsubst` Requires Exported Variables

`docker-entrypoint.sh` in the DWV container sets `API_URL` as a shell variable. If `API_URL` is not already in the container's exported environment, `envsubst` (a child process) sees it as unset and substitutes an empty string. The symptom is:

```
nginx: invalid number of arguments in "set" directive
```

Fix: always pass `API_URL` as an ECS task definition `environment` entry — the container runtime exports it automatically.

ALB Target Group Port Change Forces Replacement

Changing `port` on `aws_lb_target_group` forces a resource replacement. Terraform will fail if an ALB listener rule still references the old target group when it tries to delete it. Workaround: manually delete the affected listener rule via AWS CLI first, then `terraform apply` will recreate both the target group (new port) and the listener rule (pointing to new ARN).

Adding CI/CD for AWS

Currently AWS is deployed manually. To add automation (GitHub Actions recommended):

1. Add GitHub secrets: `AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY` , `AWS_ACCOUNT_ID`
2. Create `.github/workflows/deploy-aws.yml` triggered on push to `develop`
3. Steps: ECR login → build+push images → terraform apply → ECS force-deploy

See `.github/workflows/ci.yml` for the existing GCP Cloud Build trigger pattern.